Structures de données

Chaînes de caractères

1 EXERCICE Recherche d'un caractère

Q1. Écrire une fonction `e_recherche(ch)` qui prend en argument une chaîne de caractère et détermine si cette chaîne contient ou non le caractère "e" et, si c'est le cas, qui retourne le nombre de "e". Vous ne devez pas utiliser les fonctions préexistantes.

Q2. Essayer votre fonction sur la chaîne : `"Le vieux fermier cueille sereinement des cerises et des fraises entre les arbres de son verger"`.

Q3. Quelle est la fonction préexistante qui joue le rôle de notre fonction `e_recherche(ch)` ?

2 EXERCICE Palindrome

Un palindrome est un mot qui se lit aussi bien de gauche à droite que de droite à gauche. Par exemple : « elle », « ici », « ressasser », « radar », etc.

Écrivez une fonction `palindrome(mot)` qui retourne `1` si le mot donné en paramètre est un palindrome, et `0` dans le cas contraire.

3 EXERCICE Caractère d'espacement

Écrivez une fonction `insertion(ch)` qui recopie `ch` (dans une nouvelle variable) en insérant des tirets entre les caractères. Ainsi, par exemple, `ch="Lyon Nice"` deviendra `ch_insertion = "L-y-o-n- -N-i-c-e"`.

Vous ne devez pas utiliser les fonctions préexistantes.

4 EXERCICE Rectangle

Écrire une fonction qui prend en paramètres un caractère (par exemple `"X"`) et deux entiers, `largeur` et `hauteur`, et qui affiche un rectangle de dimension `largeur` × `hauteur`, entièrement rempli du caractère choisi.

console ×

```
In [1]: rectangle("X",4,3)
XXXX
XXXX
XXXX
```

5 EXERCICE Sapin

Proposer une fonction prenant un entier positif non nul `h` indiquant une hauteur et affichant, dans la sortie standard, un « sapin », constitué d'un motif de forme triangulaire fait du caractère `^` et d'un tronc constitué de deux `H`.

Par exemple, pour `h=4`, on souhaite obtenir le motif suivant :

console ×

```
   ^
   ^
  ^^^
 ^^^^^
^^^^^^^
   H
   H
```

Dictionnaires

6 EXERCICE Polynôme

Un polynôme peut être représenté par un dictionnaire, dont les clés sont les degrés des différents termes, et les valeurs les coefficients correspondants.

Par exemple, soit les deux polynômes $P_1 = 4$ et $P_2 = -X^5 + 2X^3 - 3$:

console ×

```
In [1]: P1 = {0:4}
In [2]: P2 = {0:-3, 3:2, 5:-1}
```

On adopte la convention que le polynôme nul est représenté par le dictionnaire vide `{}`.

Q1. Écrire une fonction `eval_poly(P, a)` qui prend en argument un polynôme `P`, représenté par un dictionnaire, et une valeur `a`, et qui renvoie la valeur $P(a)$.

console ×

```
In [3]: eval_poly(P1,3)
Out[3]: 4

In [4]: eval_poly(P2,-1)
Out[4]: -4

In [5]: eval_poly(P2,2)
Out[5]: -19
```

Q2. Écrire une fonction `derivee(P)` qui aura pour argument le dictionnaire `P` caractérisant un polynôme P, et qui renverra le dictionnaire associé au polynôme P', représentant sa dérivée.

console ×

```
In [6]: derivee(P1)
Out[6]: {}

In [7]: derivee(P2)
Out[7]: {2: 6, 4: -5}
```

Q3. Écrire alors une fonction `deriv_k(P,k)` qui aura pour argument le dictionnaire `P` caractérisant un polynôme P, et qui renverra le dictionnaire associé au polynôme $P^{(k)}$, représentant sa dérivée k-ième.

console ×

```
In [8]: deriv_k(P1,0)
Out[8]: {0: 4}

In [9]: deriv_k(P2,2)
Out[9]: {1: 12, 3: -20}

In [10]: deriv_k(P2,3)
Out[10]: {0: 12, 2: -60}
```

Listes

Dans toute la suite, il est interdit d'utiliser les fonctions ou opérateurs préexistants qui feraient le travail à notre place (l'opérateur `in`, les méthodes `.index()`, `.count()`, les fonctions `max`, `sum`, etc.) : le but de ces exercices est précisément de les réimplémenter soi-même.

7 EXERCICE
Appartenance à une liste

Écrire une fonction testant l'appartenance d'un objet à une liste. On en proposera une version à l'aide d'une boucle `for`, et une autre à l'aide d'une boucle `while`.

appartient.py ×

```
1 def appartient(x, L):
2     ...
```

console ×

```
In [1]: appartient(42, [5, 12, 17, 42
   ])
Out[1]: True

In [2]: appartient(24, [5, 12, 17, 42
   ])
Out[2]: False
```

8 EXERCICE
Positions d'un objet dans une liste

Écrire une fonction renvoyant la liste (éventuellement vide) de toutes les positions auxquelles un objet apparaît dans une liste.

positions.py ×

```
1 def positions(x, L):
2     ...
```

console ×

```
In [1]: positions(42, [12, 17, 42, 5
   ])
Out[1]: [2]

In [2]: positions(24, [12, 17, 42, 5
   ])
Out[2]: []

In [3]: positions(42, [42, 12, 17, 42
   , 5])
Out[3]: [0, 3]
```

9 EXERCICE
Maximum d'une liste

Q1. Pour une liste **dont les éléments sont des entiers**, écrire une fonction renvoyant le *maximum* des éléments de cette liste. Écrire de même une fonction renvoyant la *position* de ce maximum.

console

```
In [1]: L1 = [5, 12, 42, 7, 42,
   3]
In [2]: maximum(L1),
   position_maximum(L1)
Out[2]: (42, 2)
```

Q2. Que se passe-t-il lorsque le maximum apparaît plusieurs fois dans la liste ? Comment modifier le code pour changer ce comportement si besoin ?

10 EXERCICE
Sommes cumulées

Q1. Écrire une fonction prenant en entrée une liste de nombres, et renvoyant la liste des sommes cumulées (c'est-à-dire, à chaque position, la somme de tous les termes qui la précèdent, elle incluse) :

console

```
In [1]: L2 = [4, 6, 2, 8, 5, 3,
   9, 1, 7]
In [2]: sommes_cumulees(L2)
Out[2]: [4, 10, 12, 20, 25, 28,
   37, 38, 45]
```

Q2. Évaluer le nombre d'additions réalisées par votre fonction, en fonction de la longueur n de la liste. Si ce nombre est de l'ordre de n^2, essayer de réécrire la fonction pour arriver à un nombre d'additions de l'ordre de n.

11 EXERCICE
Plus longue sous-liste croissante

On appelle *sous-liste* d'une liste `L` (de longueur `n`) toute liste de la forme `[L[n0], L[n1], ..., L[nk]]`, où $0 \leqslant n_0 < n_1 < \cdots < n_k < n$. Elle est dite *croissante* si de plus `L[n0]` $\leqslant$ `L[n1]` $\leqslant \cdots \leqslant$ `L[nk]` ; par convention, une liste réduite à un seul élément est toujours croissante.
On cherche ici un algorithme efficace pour déterminer la plus longue sous-liste croissante d'une liste donnée. L'idée consiste à construire une nouvelle liste `M` de même longueur que `L`, où `M[i]` désigne la longueur de la plus longue sous-liste croissante de `L` **se terminant** en position `i` :

- si aucun terme avant `L[i]` ne lui est inférieur ou égal, alors `M[i] = 1` (la sous-liste réduite à `L[i]` seul);
- sinon, `M[i]` vaut $1 + \max($`M[j]`$)$, le maximum portant sur tous les indices `j < i` tels que `L[j] <= L[i]`.

Q1. Comment obtenir la longueur d'une plus longue sous-liste croissante de `L` à partir de `M` ?

Q2. Écrire une fonction `maxLongSL` qui renvoie la longueur d'une plus longue sous-liste croissante d'une liste `L` donné en entrée.

On s'autorise à utiliser la fonction `max` sur les listes.

Q3. Écrire une fonction `maxSL` renvoyant une sous-liste de taille maximale d'une liste `L` donné en argument.